# PVCM — Production Version Control Module

## PVCM — Production Version Control Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: SDPS — Software Deployment Standard

Category: Governance & Enforcement

Subcategory: Software Deployment Governance

Type: Core Module

Governed Space: Software Production Version Governance

Module ID: SLGA-SDPS-PVCM-03

Origin ID: OOF-SLGA-SDPS-PVCM

Version: 1.0

Status: Canonical Open

Effective Date: 14.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: English

## Canonical Definition

PVCM — Production Version Control Module is the Core Module within SDPS that governs how software versions actually present, activated, or operating within defined production environments are identified, associated with authorized releases and deployment records, verified against their underlying artifacts and configurations, monitored for version inconsistency, and maintained as traceable production-state references throughout deployment transitions.

## PVCM establishes production version identity as an evidence-supported

representation of operational software reality rather than a version label, deployment intention, or pipeline completion claim.

## It governs the relationship between authorized release identity, deployed

artifact identity, target environment, active production instances, configuration and dependency references, version transitions, and observed production state.

The module recognizes that one software product may operate through multiple versions simultaneously across clusters, regions, devices, services, or operational segments.

## It distinguishes a release authorized for deployment from an artifact

actually present in production, a deployed artifact from an activated version, and a reported version from an independently supported production identity.

## PVCM applies to human-operated deployments, automated pipelines,

cloud-native systems, distributed applications, embedded software, autonomous software systems, and robotic platforms.

Its principal governance output is the Software Production Version Profile .

## Operational Role

## PVCM establishes and maintains the identity and consistency of software

versions within production environments.

It governs:

• Production version identity

• Production environment association

• Release-to-production traceability

• Artifact identity verification

• Build and configuration references

• Deployment authorization association

• Installed and activated version distinctions

• Runtime version identification

• Multi-version production states

• Version distribution across operational segments

• Production version transitions

• Configuration and dependency consistency

• Version drift detection

• Unknown and conflicting version states

• Version observation evidence

• Production version reassessment

• Historical version continuity

• Handoff to deployment-state monitoring

PVCM does not authorize deployments, execute deployment procedures, establish overall service health, or independently govern rollback and recovery decisions.

## Operational Question

How can software versions actually present, activated, and operating across production environments remain identifiable, traceable, and consistent with applicable release, deployment, artifact, configuration, and operational conditions when automated deployments, distributed infrastructure, mixed-version states, and autonomous execution may cause the observed production reality to differ from the intended or authorized version?

## Position in Architecture Governance Flow

## SLGA™ Governance Flow

## Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

## Cross-cutting: Software Lifecycle Control

## SDPS Core Module Flow

## DPM — Deployment Planning Module → DAM — Deployment Authorization

Module → PVCM — Production Version Control Module → DSMM — Deployment State Monitoring Module → RRVM — Rollback & Recovery Module

PVCM is the third Core Module within SDPS.

## It receives the intended production state from DPM and the applicable

deployment authorization references from DAM.

It establishes which software versions and artifacts are actually present within the governed environment.

## Its outputs support DSMM in evaluating deployment-state consistency and

RRVM in identifying rollback or recovery targets.

• Governed Space

## Software Production Version Governance

## This governed space concerns the identification, verification,

classification, transition, and historical preservation of software version states within operational production environments.

It begins when a software artifact is introduced, detected, or proposed for activation within a production environment.

## It continues while the artifact remains present, activated, operating,

transitioning, superseded, or otherwise relevant to the environment's software state.

## It includes version identity discrepancies discovered after deployment

execution and version states resulting from partial deployments, recovery actions, or unauthorized modifications.

## Governed Object

The primary Governed Object is:

## A Software Production Version State representing the evidence-supported

association between an identifiable software artifact, its release and deployment references, a defined production environment or operational segment, its installation or activation condition, and its observed version identity at a specified time.

The governed object must preserve:

## Production Version ID

## Software Product ID

## Release ID

## Artifact Identity

## Build Reference

## Deployment ID

## Deployment Authorization Reference

## Environment ID

## Operational Segment ID

## Configuration Reference

## Dependency Reference

## Installation State

## Activation State

## Observed Version State

## Observation Time

## Evidence Reference

Version

## Scope

PVCM governs:

## Production Version Identification

## Release Association

## Artifact Identity Verification

## Build Reference Association

## Environment and Segment Identification

## Installation and Activation State

## Runtime Version Observation

## Multi-Version Representation

## Version Distribution

## Production Version Consistency

## Configuration and Dependency References

## Version Drift

## Version Conflicts

## Version Transition Traceability

## Version Evidence

## Historical State Preservation

## Reassessment

## Monitoring Handoff

PVCM does not prescribe a particular versioning convention, deployment tool, container platform, or software packaging technology.

## Foundational Principle

The software version governing production reality is the version supported by evidence of what is actually present and operating—not merely the version that was intended, approved, labeled, or reported as deployed.

Release Version ≠ Production Version.

## Deployment Reported Successful ≠ Authorized Artifact Verified in

Production.

Version Label Equality ≠ Artifact Identity Equality.

Artifact Installed ≠ Artifact Activated.

Artifact Activated ≠ Artifact Operating as Intended.

## Production Version Identity

Every governed production version must have an identifiable reference.

Relevant elements include:

## Production Version ID

## Software Product ID

## Release ID

## Artifact Identity

## Environment ID

## Operational Segment ID

## Deployment Reference

## Version State

## Observation Time

## Evidence Reference

A production version identity must remain distinguishable from a software release identifier and a deployment execution identifier.

## Production Environment Association

PVCM must identify the operational environment in which a software version is present.

Relevant elements include:

## Environment ID

## Environment Type

## Infrastructure Reference

## Operational Purpose

## Environment Boundary

## Relevant Services

## Version Observation Scope

## Environment State Reference

For distributed systems, environment identification may require multiple operational segments rather than one undifferentiated production label.

## Operational Segment Identity

An operational segment is an identifiable portion of a production environment relevant to version governance.

Examples include:

## Service Instance

## Container

## Virtual Machine

## Physical Server

## Cluster

## Region

## Availability Zone

## Edge Device

## Embedded Controller

## Robotic Unit

## Device Fleet Group

PVCM must identify segments at the level necessary to represent material production version differences.

## Release-to-Production Association

PVCM must preserve the relationship between a production artifact and its applicable software release.

Relevant elements include:

## Release ID

## Release Record Reference

## Release Authorization Reference

## Authorized Artifact Identity

## Deployment ID

## Observed Artifact Identity

## Production Version State

A production artifact must not be assumed to belong to an authorized release solely because its version label matches the release label.

## Deployment Authorization Association

PVCM must preserve the applicable deployment authorization reference where available.

Relevant elements include:

## Deployment Authorization ID

## Authorized Release

## Authorized Environment

## Authorized Scope

## Authorized Actor Reference

## Authorization Validity

## Authorization Restrictions

## Observed Deployment Association

PVCM does not determine the legitimacy of deployment authorization.

DAM governs that determination.

PVCM identifies whether the observed production version can be associated with the relevant authorization.

## Artifact Identity

PVCM requires production artifact identity to be established through suitable evidence.

Relevant identifiers may include:

## Cryptographic Digest

## Artifact Registry Identity

## Signed Artifact Reference

## Build Provenance Reference

## Package Identity

## Container Image Digest

## Binary Identity

## Firmware Identity

## Immutable Artifact Reference

## The appropriate method depends on the software artifact and deployment

environment.

A human-readable version string alone may be insufficient.

## Artifact Identity Verification

## Artifact identity verification evaluates whether the observed production

artifact corresponds to the expected artifact.

Relevant elements include:

## Expected Artifact Identity

## Observed Artifact Identity

## Verification Method

## Verification Time

## Evidence Source

## Verification Result

## Known Limitations

## Conflicting Evidence

Where direct artifact verification is unavailable, the limitation must remain explicit.

Artifact Reference Available ≠ Artifact Identity Verified.

## Build Identity Association

PVCM must preserve relevant build identity references established through SCFS.

These may include:

## Build ID

## Build Configuration

## Source Reference

## Build Environment Reference

## Build Provenance

## Build Reproducibility Reference

## Artifact Output Identity

PVCM does not reproduce build governance.

## It evaluates the association between the build reference and the artifact

actually present in production.

## Version Label Governance

Version labels may support identification but must not replace artifact identity evidence.

Examples include:

## Semantic Version

## Release Number

## Build Number

## Commit Reference

## Container Tag

## Firmware Version

## Deployment Revision

## Internal Version Identifier

Labels may be reused, overwritten, incorrectly assigned, or applied to different artifacts.

Version Label Match ≠ Verified Artifact Match.

## Intended Production Version

## PVCM receives the intended production version from DPM and relevant

deployment references.

Relevant elements include:

## Target Release

## Target Artifact

## Target Environment

## Target Operational Segments

## Target Configuration

## Target Dependency State

## Intended Activation State

## Planned Version Distribution

The intended version provides a comparison reference.

It does not establish the actual production version.

## Observed Production Version

## The observed production version represents the software identity supported

by evidence from the operational environment.

Relevant evidence may include:

## Artifact Inspection

## Runtime Identity Reporting

## Container Image Digest

## Installed Package Metadata

## Firmware Attestation

## Deployment Inventory

## Process or Service Identity

## Trusted Agent Observation

## Operational State Record

Observed identity must preserve the observation source, time, scope, and limitations.

## Intended–Observed Version Alignment

PVCM evaluates alignment between intended and observed production versions.

Relevant relationships include:

## Intended Release ↔ Observed Release

## Intended Artifact ↔ Observed Artifact

## Intended Environment ↔ Observed Environment

## Intended Segment ↔ Observed Segment

## Intended Configuration ↔ Observed Configuration

## Intended Activation ↔ Observed Activation

Material misalignment must be classified and supplied to the relevant deployment governance mechanisms.

## Installation State

PVCM must distinguish software presence from software activation.

Relevant installation states include:

## Not Present

## Transfer Pending

## Transferred

## Installation In Progress

## Installed

## Partially Installed

## Installation Failed

## Installation State Unknown

Installation state must remain associated with the relevant artifact and operational segment.

## Activation State

## Activation state concerns whether an installed artifact has been enabled

for its intended operational function.

Relevant states include:

## Not Activated

## Activation Pending

## Activation In Progress

## Activated

## Partially Activated

## Activation Failed

## Deactivated

## Activation State Unknown

Activation may be controlled through service startup, traffic routing, feature flags, firmware switching, or other mechanisms.

## Runtime Version State

Runtime version state concerns which software artifact is actually executing or serving the relevant operational function.

Relevant states include:

## Running

## Partially Running

## Standby

## Inactive

## Degraded

## Terminated

## Runtime State Unknown

## PVCM governs runtime identity, not the complete operational quality or

health of the running software.

DSMM evaluates the broader deployment-state implications.

## Installed–Activated–Running Distinction

PVCM must preserve three separate questions:

What software is installed?

What software is activated?

What software is actually running or serving the operational function?

These conditions may differ.

For example, a new artifact may be installed but inactive while the previous version continues serving production traffic.

A service may report the new version as activated while some instances still execute the previous artifact.

Installed ≠ Activated ≠ Running.

## Multi-Version Production State

PVCM recognizes that multiple software versions may coexist legitimately.

Examples include:

## Rolling Deployments

## Canary Releases

## Blue–Green Environments

## Regional Rollouts

## Mixed-Version Clusters

## Edge Device Fleets

## Gradual Firmware Updates

## Compatibility Transition Periods

Multi-version operation must remain identifiable rather than being treated automatically as a version conflict.

## Version Distribution

## Version distribution represents the relationship between software versions

and the operational segments where they are present or active.

Relevant elements include:

## Version Identity

## Segment Identity

## Installation State

## Activation State

## Runtime State

## Observation Time

## Distribution Scope

## Evidence Coverage

Where meaningful, distribution may be expressed through counts, proportions, regions, clusters, or device groups.

Distribution claims must remain supported by the available observation coverage.

## Canary Version Governance

Canary deployments introduce a new version into a limited operational scope.

PVCM must identify:

## Canary Version

## Canary Segments

## Existing Production Version

## Canary Scope

## Activation State

## Observed Version Distribution

## Expansion References

## Version Evidence

PVCM does not independently determine whether the canary's operational performance justifies broader rollout.

DSMM governs the relevant operational-state assessment.

## Rolling Version Transition

A rolling deployment may create temporary mixed-version states.

PVCM must preserve:

## Previous Version

## Target Version

## Affected Segments

## Transition Sequence Reference

## Current Version Distribution

## Completed Segments

## Pending Segments

## Failed Segments

## Transition Evidence

A rolling deployment must not be classified as uniformly updated when material segments remain on earlier versions.

## Blue–Green Version State

Blue–green deployments may maintain two production-capable environments.

PVCM must distinguish:

## Blue Environment Identity

## Green Environment Identity

## Artifact Identity in Each Environment

## Activation State

## Traffic-Serving State

## Version Transition Reference

## Observed Production Version

## Evidence

The environment containing the newer artifact is not necessarily the environment currently serving production traffic.

## Feature-Gated Version State

Software functionality may be activated or restricted through feature flags or equivalent mechanisms.

PVCM must distinguish:

## Artifact Version

## Installed Version

## Activated Feature Configuration

## Operational Segment

## Feature State Reference

## Observation Evidence

Feature configuration may materially affect the operational meaning of a production version.

PVCM records the relevant version and configuration association.

## Broader behavioral and operational assessments remain governed by

applicable modules.

## Production Configuration Association

PVCM must preserve the configuration references relevant to production version identity.

Relevant elements include:

## Configuration Baseline

## Applied Configuration

## Environment-Specific Configuration

## Runtime Configuration Reference

## Configuration Version

## Observed Configuration State

## Known Configuration Difference

## Evidence

Configuration governance remains within SCFS.

PVCM governs the production-version relevance of those references.

## Dependency State Association

PVCM must preserve relevant dependency references associated with the observed production version.

These may include:

## Dependency Baseline

## Runtime Dependency Versions

## Container Base Image

## Linked Libraries

## External Service Dependencies

## Firmware Dependencies

## Dependency Restrictions

## Observed Dependency Differences

Dependency governance remains within SDS.

## PVCM identifies deployment-relevant dependency associations and

discrepancies.

## Production Version Consistency

## Production version consistency concerns whether the observed production

state aligns with the applicable release, deployment, artifact, configuration, and environment references.

Relevant dimensions include:

## Release Consistency

## Artifact Consistency

## Build Consistency

## Environment Consistency

## Configuration Consistency

## Dependency Consistency

## Activation Consistency

## Version Distribution Consistency

Consistency must be assessed within the relevant operational scope and observation time.

## Version Consistency Classification

PVCM applies the following consistency conditions:

## Consistent

Consistent Within Authorized Mixed-Version Scope

## Inconsistent

## Partially Verified

## Unknown

## Conflicted

## Not Applicable

A mixed-version environment may be consistent when the distribution matches an authorized transition plan.

Conversely, a single-version environment may be inconsistent if the observed artifact differs from the authorized artifact.

## Production Version Drift

Version drift occurs when the observed production version state materially diverges from its applicable reference state.

Potential causes include:

## Unauthorized Deployment

## Partial Rollout

## Failed Update

## Manual Artifact Replacement

## Mutable Container Tag

## Unexpected Rollback

## Configuration Change

## Dependency Change

## Device Update Failure

## Unrecorded Infrastructure Action

PVCM must identify the observed deviation without assuming its cause.

## Drift Detection

## Version drift detection requires comparison between applicable reference

and observed states.

Relevant elements include:

## Reference Version State

## Observed Version State

## Comparison Scope

## Observation Time

## Detection Method

## Difference

## Materiality

## Evidence

## Required Reassessment Reference

A discrepancy must not be dismissed solely because the software remains operational.

## Unauthorized Production Version

A production artifact may be present without an applicable deployment authorization.

PVCM must preserve:

## Observed Artifact Identity

## Environment

## Operational Segment

## Observed State

## Deployment Reference, If Available

## Authorization Reference, If Available

## Evidence

## Authorization Gap

## PVCM identifies the production version and the authorization association

gap.

DAM determines authorization legitimacy.

Artifact Present in Production ≠ Artifact Legitimately Deployed.

## Unknown Production Version

An unknown production version exists when available evidence cannot sufficiently establish the artifact identity or relevant version state.

Examples include:

## Unverifiable Binary

## Missing Artifact Provenance

## Unavailable Runtime Inventory

## Inconsistent Version Reports

## Unidentified Firmware

## Unobserved Production Segment

## Incomplete Deployment Records

Unknown identity must not be replaced with an assumed release version.

## Conflicting Production Version Evidence

Version evidence may conflict.

Examples include:

## Deployment Pipeline Reports Version A

## Runtime Reports Version B

## Artifact Digest Corresponds to Version C

## Inventory Contains an Outdated Record

PVCM must preserve the conflict and identify the affected production scope.

The current version must not be classified as verified until the material conflict is resolved or legitimately bounded.

## Production Version Observation

PVCM must preserve how version information was obtained.

Relevant elements include:

## Observation ID

## Observation Source

## Observed Artifact

## Environment

## Operational Segment

## Observation Time

## Verification Method

## Evidence Reliability

## Known Limitations

## Observed Result

Observation must remain distinguishable from deployment intention and execution reports.

## Observation Coverage

## Observation coverage concerns how much of the relevant production

environment is represented by available version evidence.

Relevant dimensions include:

## Environment Coverage

## Service Coverage

## Instance Coverage

## Regional Coverage

## Device Coverage

## Temporal Coverage

## Artifact Verification Coverage

## Unobserved Segments

A claim that all production instances run a specified version requires evidence sufficient for the claimed scope.

## Observation Freshness

Production version observations may become outdated.

PVCM must identify:

## Observation Time

## Relevant Deployment Activity

## Subsequent Version Changes

## Environment Changes

## Expected Observation Validity

## Reverification Requirements

Historical version evidence must not automatically be treated as proof of the current production state.

## Evidence Provenance

Production version evidence must remain attributable to its source.

Relevant elements include:

## Evidence ID

## Evidence Source

## Collection Method

## Collection Actor or System

## Collection Time

## Observed Target

## Integrity Reference

## Verification Result

## Known Limitations

## Evidence provenance supports reconstruction and assessment of version

identity claims.

## Evidence Sufficiency

PVCM must assess whether the available evidence supports the claimed production version state.

Relevant dimensions include:

## Artifact Identity Evidence

## Environment Identity Evidence

## Operational Segment Coverage

## Installation State Evidence

## Activation State Evidence

## Runtime Identity Evidence

## Configuration Reference Evidence

## Dependency Reference Evidence

## Evidence Freshness

## Evidence Consistency

Evidence Available ≠ Production Version Verified.

## Production Version Transition

A production version transition occurs when the version state of an operational environment or segment changes.

Relevant transitions include:

## Installation

## Activation

## Deactivation

## Version Replacement

## Traffic Switch

## Rolling Update

## Rollback

## Recovery Replacement

## Environment Migration

## Version Removal

PVCM must preserve the relevant before-and-after version identities and transition references.

## Transition Traceability

Production version transitions must remain reconstructable.

Relevant elements include:

## Transition ID

## Previous Version

## Target Version

## Observed Resulting Version

## Environment

## Operational Segment

## Deployment or Recovery Reference

## Transition Time

## Execution Evidence Reference

## Verification Evidence

A transition record must not automatically be treated as proof that the intended resulting version was established.

## Partial Version Transition

A version transition may complete in some operational segments while remaining incomplete in others.

PVCM must identify:

## Completed Segments

## Pending Segments

## Failed Segments

## Unknown Segments

## Previous Versions

## Target Versions

## Observed Distribution

## Evidence Coverage

Partial transitions must not be collapsed into a single completed state when material version differences remain.

## Version Transition Failure

Version transition failure may include:

## Artifact Installation Failure

## Activation Failure

## Version Identity Mismatch

## Incomplete Rollout

## Unexpected Version Reversion

## Configuration Incompatibility

## Dependency Incompatibility

## Unknown Resulting Version

PVCM governs the identity and version-state consequences.

## DSMM evaluates broader deployment-state consequences, and RRVM governs

applicable rollback or recovery.

## Version Reassessment

Production version reassessment may be required when:

## New Deployment Evidence Emerges

## Artifact Identity Changes

## Runtime Reports Conflict

## Configuration Changes

## Dependencies Change

## New Operational Segments Appear

Unauthorized Modification Is Detected

## Rollback Occurs

## Recovery Changes the Production State

## Observation Evidence Becomes Stale

Reassessment must preserve the earlier supported state and the reason for reclassification.

## Historical Production Version Continuity

PVCM must preserve relevant historical version states.

These may include:

## Previously Installed Version

## Previously Activated Version

## Previously Running Version

## Superseded Version

## Rolled-Back Version

## Recovered Version

## Removed Version

## Unknown Historical Version

Historical records must remain distinguishable from current observations.

## Production Version Governance Classification

PVCM recognizes the following classifications:

## Version Not Established

## Version Identification Pending

### Artifact Present

### Installed

### Activation Pending

### Activated

### Running

### Standby

### Mixed-Version State

### Version Transition In Progress

### Version Partially Verified

### Version Verified

### Version Inconsistent

### Version Drift Detected

### Artifact Identity Conflict

### Authorization Association Missing

### Observation Insufficient

### Reassessment Required

### Superseded

### Removed

### Unknown

### Conflicted

A classification must identify the relevant environment, segment, and observation time.

## Governance Requirements

A conforming PVCM implementation must satisfy the following requirements. Identity and Association

• Establish production version identity

• Identify the software product

- Preserve release association
- Preserve artifact identity
- Identify the production environment
- Identify relevant operational segments
- Preserve deployment references
- Preserve applicable authorization references
- Preserve build identity references
- Preserve configuration and dependency references

## Production State

- Distinguish installed, activated, and running states
- Identify the intended production version
- Identify the observed production version
- Preserve observation time
- Represent multi-version environments
- Preserve version distribution
- Identify material partial transitions
- Preserve version transition references
- Distinguish version identity from operational health
- Preserve historical version states

## Verification and Consistency

- Establish suitable artifact verification methods
- Preserve verification results
- Compare intended and observed artifact identities
- Evaluate version consistency
- Identify material configuration differences
- Identify material dependency differences
- Detect production version drift
- Preserve unknown version states
- Preserve conflicting version evidence
- Prevent version labels from replacing artifact verification

## Evidence and Continuity

- Preserve version observation evidence
- Identify evidence provenance
- Evaluate observation coverage

• Evaluate evidence freshness

• Evaluate evidence sufficiency

• Preserve material evidence limitations

• Identify reassessment triggers

• Preserve version-state revisions

• Maintain production version traceability

• Provide version references to deployment-state monitoring

## Governance Logic

## Authorized Release Reference → Intended Production Version → Target

Environment → Operational Segments → Deployment Execution Reference → Artifact Observation → Installation State → Activation State → Runtime Version State → Artifact Identity Verification → Configuration and Dependency Association → Intended–Observed Comparison → Version Consistency Classification → Production Version Profile

Where multiple versions coexist:

## Environment → Operational Segments → Observed Versions → Version

Distribution → Authorized Transition Reference → Mixed-Version Consistency Assessment

Where discrepancies occur:

## Reference Version → Observed Version → Material Difference → Evidence

Assessment → Consistent / Inconsistent / Unknown / Conflicted → Reassessment or Intervention Reference

PVCM must not convert deployment execution reports into verified production identity without sufficient evidence.

## Governance Outputs

The principal output is:

## Software Production Version Profile

Supporting outputs may include:

## Production Version Identity

## Release-to-Production Association

## Artifact Verification Record

## Build Identity Reference

## Production Environment Reference

## Operational Segment Register

## Installed Version Record

## Activated Version Record

## Runtime Version Record

## Version Distribution Map

## Multi-Version State Record

## Production Configuration Reference

## Production Dependency Reference

## Version Consistency Assessment

## Version Drift Record

## Version Conflict Record

## Version Observation Evidence

## Evidence Coverage Assessment

## Production Version Transition Record

## Historical Production Version Register

## Production Version Governance Classification

## Deployment Monitoring Handoff

## Minimum Implementation Framework

A minimum PVCM implementation must establish eight governance layers. Layer 1 — Production Identity

## Identify the software product, release, production version, environment,

and relevant operational segments. Layer 2 — Artifact Association

## Preserve artifact identity, build references, deployment references, and

applicable authorization associations. Layer 3 — Operational Version State

Distinguish installed, activated, and running versions within the relevant environment. Layer 4 — Multi-Version Representation

## Represent material version differences across services, instances, regions,

clusters, devices, or other operational segments. Layer 5 — Consistency Verification

## Compare intended and observed production versions, artifacts,

configurations, and relevant dependency states. Layer 6 — Evidence

## Preserve observation methods, sources, times, coverage, freshness,

integrity references, and limitations. Layer 7 — Transition and Continuity

## Maintain version transitions, historical states, drift findings, conflicts,

and reassessment requirements. Layer 8 — Governed Output and Handoff

## Establish the Software Production Version Profile and provide relevant

version-state references to DSMM and other applicable governance mechanisms.

The implementation may use manual inventories, software agents, runtime inspection, deployment platforms, artifact registries, or other suitable mechanisms.

The governance logic remains independent of the chosen technology.

## Cross-Module Boundary

## DPM → PVCM

DPM defines the intended production version and target operational state.

PVCM identifies the versions actually present, activated, or running.

## DAM → PVCM

DAM establishes the legitimate deployment authorization.

## PVCM preserves the association between observed production versions and

applicable authorization references.

## PVCM → DSMM

PVCM establishes production version identity and consistency.

## DSMM evaluates the broader deployment state, operational conditions,

monitoring evidence, and intervention triggers.

## PVCM → RRVM

PVCM identifies previous, current, and resulting production versions.

## RRVM governs rollback and recovery actions and verifies their operational

consequences.

Production Version Verified ≠ Deployment Operationally Acceptable.

## Cross-Architecture Boundary

PVCM consumes relevant references from:

## SRLS — Software Release Standard

Release identity, artifact references, authorization, and records.

## SCFS — Software Configuration Standard

## Build identity, artifact version identification, configuration baselines,

and configuration change references.

## SDS — Software Dependency Standard

Dependency identity, version, suitability, and continuity references.

## SSS — Software Security Standard

Security-relevant artifact and dependency conditions.

## SLCS — Software Lifecycle Control Standard

Lifecycle state and transition references.

PVCM does not replace release, build, configuration, dependency, security, or lifecycle control governance.

## It governs the production-state identity and consistency of the software

artifacts to which those mechanisms relate.

## Practical Applications

## Cloud-Native Production Systems

## Identifying actual container images and software versions across clusters,

regions, and service instances.

## Continuous Delivery

Verifying that the artifact running in production corresponds to the authorized release rather than relying on pipeline success reports.

## Autonomous Software Deployment

Detecting version mismatches or unrecorded changes introduced through autonomous agent execution.

## Industrial and Embedded Software

## Preserving firmware identity across equipment, controllers, and distributed

device fleets.

## Robotics

Identifying which software versions are operating on physical robotic units, including partially updated fleets.

## Critical Infrastructure

## Maintaining evidence-supported production version identities for

operational systems where unknown or inconsistent versions create significant risks.

## Use Case 1 — The Pipeline Reports the Correct Version but Production

Runs a Different Artifact

A software provider authorizes release 4.2 for production deployment.

The deployment pipeline reports successful installation of version 4.2.

However, the container tag used during deployment was mutable.

The tag now points to an artifact with a different cryptographic digest.

PVCM evaluates:

## Authorized Release → Expected Artifact Identity → Deployment Reference →

Observed Production Artifact → Digest Comparison → Version Consistency

The assessment identifies:

• The displayed version label matches the expected release

• The observed artifact identity differs from the authorized artifact

• The deployment completion report does not establish artifact

• The production version cannot be classified as verified against the

consistency. authorized release.

The appropriate classification is:

Version Inconsistent — Artifact Identity Mismatch.

Version Label Equality ≠ Artifact Identity Equality.

## Use Case 2 — A Rolling Deployment Leaves an Unidentified Mixed-Version

Production State

A cloud service deploys a new software version across multiple clusters.

Most clusters receive the update.

Several instances fail during deployment and remain on the previous version.

The deployment dashboard reports overall completion.

PVCM evaluates:

## Target Version → Cluster Identity → Instance Identity → Observed Version

Distribution → Transition Evidence → Consistency Assessment

The assessment identifies:

• The intended version is identifiable

• Multiple operational segments remain on the previous version

• The actual distribution differs from the intended completed state

• The overall deployment completion claim is not supported for the full

environment.

The appropriate classification is:

Mixed-Version State — Partial Transition.

Deployment Reported Complete ≠ Every Production Segment Updated.

## Use Case 3 — A Robotic Fleet Contains Unknown Firmware Versions

A robotics operator manages a fleet of autonomous mobile robots.

A firmware update is deployed through a remote management system.

Most robots report the expected firmware version.

Several disconnected units provide no current version evidence.

The management platform displays the update campaign as completed.

PVCM evaluates:

## Firmware Release → Device Fleet → Device Identity → Observed Firmware

Version → Observation Coverage → Evidence Freshness

The assessment identifies:

• The intended firmware release is identifiable

• Most devices provide supporting version evidence

• Several devices have no current observation

• The update completion claim exceeds the verified observation coverage

• The unobserved devices must retain an unknown version state

The appropriate classification is:

Partially Verified — Unknown Device Version States.

Fleet Update Reported Complete ≠ Every Device Version Verified.

## Critical Distinctions

Release Version ≠ Production Version.

## Deployment Reported Successful ≠ Authorized Artifact Verified in

Production.

Version Label Equality ≠ Artifact Identity Equality.

Artifact Installed ≠ Artifact Activated.

Artifact Activated ≠ Artifact Operating as Intended.

Installed ≠ Activated ≠ Running.

Artifact Reference Available ≠ Artifact Identity Verified.

Version Label Match ≠ Verified Artifact Match.

Artifact Present in Production ≠ Artifact Legitimately Deployed.

Evidence Available ≠ Production Version Verified.

Historical Version Evidence ≠ Current Production Version Evidence.

Deployment Transition Recorded ≠ Resulting Version Verified.

Mixed-Version State ≠ Automatically Inconsistent State.

Production Version Verified ≠ Deployment Operationally Acceptable.

Deployment Reported Complete ≠ Every Production Segment Updated.

Fleet Update Reported Complete ≠ Every Device Version Verified.

## Canonical Boundary

## PVCM governs the identity, verification, consistency, distribution,

transition, and historical continuity of software versions actually present, activated, or operating within defined production environments.

## It establishes the evidence-supported relationship between release

identity, artifact identity, deployment references, operational segments, configuration and dependency references, and observed production version states.

PVCM does not authorize deployment, execute deployment actions, establish complete operational health, or independently govern rollback and recovery.

Its principal governance output is the:

## Software Production Version Profile

## This output provides the production identity and consistency basis for

DSMM, RRVM, and relevant software lifecycle governance mechanisms.

## Canonical Closing Statement

The software version that matters operationally is not necessarily the version that was approved, deployed, or displayed by a management system. It is the version that can be identified and verified in the environment where software actually operates.

A release may be authorized while production contains a different artifact.

A deployment pipeline may report success while some instances remain on an earlier version.

A container tag may display the expected version while referencing an unexpected binary.

A software update may be installed without being activated.

A robotic fleet may appear updated while disconnected devices retain unknown firmware states.

And a previous production version may remain active after a deployment system reports that it has been replaced.

## PVCM establishes the governance structure required to identify these

differences.

## It requires production software to remain associated with verifiable

artifacts, environments, operational segments, and observation evidence.

## It preserves the distinction between intended and observed versions,

between installation and activation, and between version identity and operational health.

## It recognizes legitimate mixed-version states without allowing partial

deployments or unknown instances to disappear behind a single product-level version label.

## It maintains version transition histories and identifies material

discrepancies requiring reassessment.

Production version governance begins where deployment intention meets operational evidence.

PVCM ensures that the software identity represented in governance records corresponds to the software reality that can actually be established in production.